

Capstone Project Milestone 1: Discovery Report

DataSage: An Agentic System for Data Quality, Cleaning, and Analysis · Group 25

Team repository: <https://github.com/shrey-Bish/cse598-group25-datasage>

Basic Information

Student name

Shrey Bishnoi

Arsha Jindal

Ritik Agarwal

Vatsal Nirmal

Maulik Jadav

Stakeholder name

Vyshanth Buddani

Email

sbishno2@asu.edu

ajinda20@asu.edu

ragarw68@asu.edu

vnirmal3@asu.edu

mjadav@asu.edu

Meeting Date

September 28, 2026

Section 1. Refined Problem Definition and Scope

Problem. Exported spreadsheets (sales exports, survey results, course records) often contain missing values, repeated rows, labels typed in different ways, broken dates and numbers that do not add up. Analyzed as they are, they give wrong answers and nothing warns the user. The stakeholder's baseline detects every planted problem in its own example file but still reports Accessories revenue as \$1,350, because a duplicate row is counted twice and the lowercase "accessories" row stays separate. After removing the duplicate and merging the labels, the total is \$830. Electronics is \$2,185 if a negative webcam row is a refund, or \$2,245 if it is an error; only the user can say which. Cleaning by hand fixes this, but it is slow and the decisions are rarely written down.

Goal and tasks. DataSage takes a messy table and a question, finds the problems that matter for that question, fixes the safe ones, asks the user about risky ones, checks that each fix did what it should, and answers the question on the cleaned data with a record of every change. To do this it profiles the table, detects problems, decides whether to fix, flag or ignore each one, applies and checks fixes on a working copy, runs the requested analysis, and writes the report.

Intended users: Analysts, students and small-business staff who are comfortable with spreadsheets but do not want to write cleaning code. They need a correct answer, and they need to be able to explain where it came from.

Use scenario: A user uploads the August sales export and asks "What was total revenue by category?" DataSage reports one duplicate row, two labels that differ only by capitalization, one unreadable date, one negative-revenue row, and one missing region that does not affect this question. It merges the labels on its own, asks before removing the duplicate or excluding the negative row, then answers and lists every change it made.

Input: one table as CSV or XLSX (one sheet, one header row, up to about 100,000 rows and 50 columns); an analysis request in plain English; and an optional short rules file describing columns and valid values, for example "Revenue = Quantity × Price" or "Region is one of North, South, East, West".

Output: (1) a quality report listing each problem with its type, column and rows, and its outcome (fixed, flagged or left alone); (2) a change log with each change's reason, rows touched and approval status; (3) a cleaned copy of the file, with the original never modified; (4) the answer as a table with a short summary and caveats; and (5) a short pandas script that replays the cleaning without the agent.

Scope: In scope: single-table CSV and Excel files; missing values, exact and near-duplicate rows, label variants (case, spaces, spelling), invalid or mixed date formats, numbers stored as text, negative or out-of-range values, and cross-column rule violations; and any analysis pandas can express (filters, group-by and time summaries, basic statistics). Out of scope: unstructured data, multi-table joins, data too large for memory, machine-learning models, and writing changes back to the source system. JSON input and retrieval of column definitions from trusted documentation are stretch goals (Table 1, item 3).

Success and failure: Success means the answer matches the answer on correctly cleaned data, every change in the cleaned file traces to a logged reason, nothing destructive happens without approval, and the user is not asked about obvious fixes. Failure means a confidently wrong number, a silent change, a made-up column or value, a fix that changes the data's meaning (e.g., dropping valid refunds), or looping without an answer. A silent wrong change is worse than an unanswered question, so when unsure DataSage flags rather than fixes.

Section 2. Stakeholder Discovery and Feedback

Meeting: September 28, 2026, 4:00 to 5:30 PM (90 minutes), online over Zoom. Attendees: Vyshanth Buddani (stakeholder) and team members Shrey Bishnoi, Arsha Jindal, Ritik Agarwal, Vatsal Nirmal and Maulik Jadav.

Stakeholder's needs, current workflow, constraints and priorities. Vyshanth sees DataSage serving students, analysts, researchers and small teams who work with CSV and Excel files, where data-quality problems are found and fixed today through slow manual review. His top priority is correctness: results must be right, the original data must never change, every modification must be logged, and uncertain changes must require approval. Only simple formatting fixes may run without asking. He asked us to keep the project focused and reliable rather than add many features: CSV and Excel and the common tabular issues first, with complex formats and machine learning left for later. Latency and cost matter less than correctness.

Table 1 lists six refinements to the original proposal and the stakeholder's decision on each. Section 5 describes our own test of the baseline.

Table 1: Refinements proposed to the stakeholder.

#	Proposed refinement	Why	Stakeholder decision
1	Clean a copy, never the original, and log every change with its reason.	Lets us check the proposal's "no unsupported data changes" criterion exactly.	Confirmed
2	Auto-apply safe formatting fixes; ask before risky ones (removing rows, filling values, changing numbers, fuzzy merges). See Table 2.	The proposal plans approval for ambiguous changes; we must agree where the line is.	Confirmed
3	Replace open retrieval of external definitions with an optional user-supplied rules file.	Online rules may not fit the data and are hard to check; a rules file is explicit and testable.	Changed. Use a user-provided rules file initially. Retrieval from trusted documentation can be a later extension if time allows.
4	Support any analysis expressible as filter, group by and aggregate, with sandboxed pandas as a fallback.	Generalizes the single fixed analysis while keeping results checkable.	Confirmed
5	Evaluate mainly on public tables with planted errors, plus a few real messy files.	Real files rarely come with correct answers; planted errors give exact ground truth.	Confirmed
6	Use the metrics and targets in Section 6 as the definition of success.	Success metrics must be agreed with the stakeholder.	Confirmed with changes. Use the proposed metrics, but treat the numerical targets as initial goals that can be adjusted after baseline testing.

Main feedback and how it changed the proposal. Vyshanth confirmed the core CSV/Excel cleaning and analysis workflow, preserving the original data, logging changes, and requiring approval for risky fixes. He preferred keeping the project focused and reliable rather than adding too many features. A user-provided rules file will be used initially, with retrieval from trusted documentation left as an optional extension. He also agreed with the proposed evaluation approach, with numerical targets treated as initial goals that can be adjusted after baseline testing.

Stakeholder confirmation. Vyshanth confirmed the refined scope, stakeholder decisions and initial success metrics during the meeting on September 28, 2026. He later reviewed this final report and approved it. Meeting notes are in Appendix A.

Section 3. Agentic System Design and Technical Approach

3.1 Why an agent, not one model call or a fixed workflow

- **One model call:** the model cannot see a whole file (100,000 rows exceed its context and raise privacy concerns), cannot add up thousands of numbers reliably, and cannot check whether its own fixes worked.
- **A fixed workflow:** this is the baseline. It runs the same checks on every file and misses anything outside its rules. On our 5-row probe file with four planted problems (eval/probes/hidden_issues.csv) it caught only one, missing a label with a trailing space, an invalid date in a column called "Day" (it only checks columns named "date" or "time"), and a revenue value not equal to quantity times price.

The right action depends on the file and the question. A missing Region does not matter for revenue by category but does for revenue by region; a negative quantity may be a refund to keep or a typo to fix. These choices need judgment, and after each fix the system must look at the result and decide what to do next. That observe-and-decide loop is why we need an agent. Our design rule: judgment in the model, all computation in ordinary code.

3.2 Workflow

Figure 1 shows the full workflow; the steps below it walk through one run.

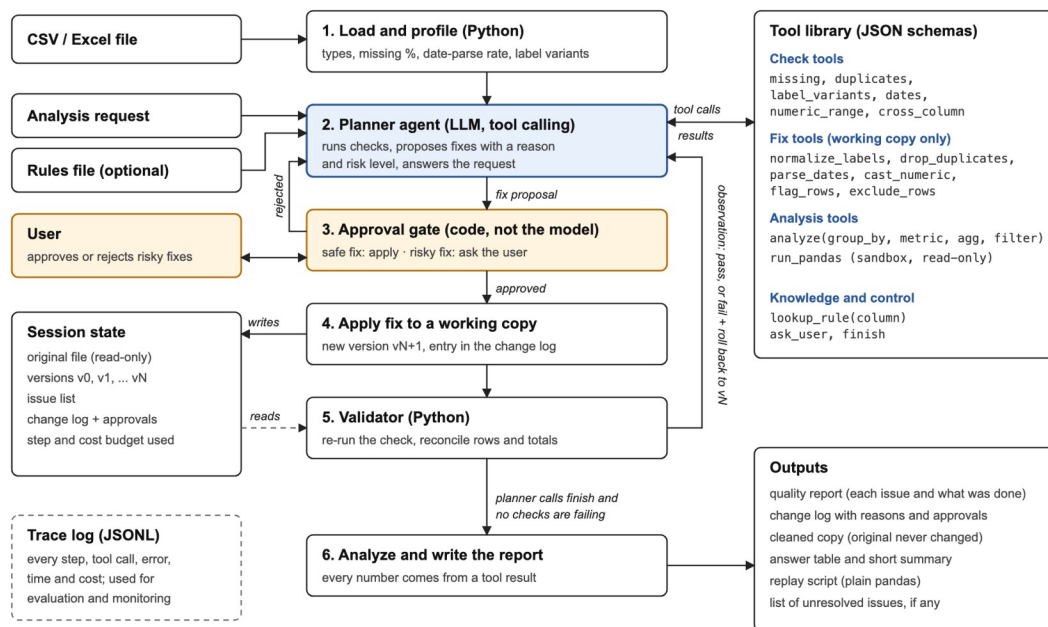


Figure 1: DataSage architecture. Blue is the only component that uses a language model; orange is where a person or the risk policy decides.

1. **Load and profile:** pandas loads the file and a profiler records each column's type, missing rate, distinct values, date and number parse rates, and label variants. The model sees only this profile and up to 20 sample rows chosen to include unusual values.
2. **Plan and check:** the planner agent reads the profile, request and rules file, then calls check tools on the columns that matter.
3. **Propose fixes:** for each problem worth fixing, the planner proposes one fix from a fixed set of fix tools, with a reason.
4. **Approval gate:** code, not the model, sets each fix's risk from its type (Table 2), so the model cannot argue past it. Safe fixes run directly; risky ones go to the user. A rejected fix becomes a flag in the report.

5. **Apply:** each fix runs on a working copy and creates a new version (v1, v2, ...) plus a change-log entry.
6. **Validate:** the validator re-runs the related check and tests invariants: rows removed match the fix, other columns are unchanged, and totals reconcile. On failure the version is rolled back and the failure returns to the planner as an observation. Steps 2 to 6 repeat.
7. **Analyze:** the planner calls the analyze tool with structured arguments (group by, metric, aggregate, filter). For anything else it writes pandas code that runs in a sandbox: a separate process with a time limit, no network and read-only data.
8. **Report:** the report builder writes the Section 1 outputs. A final check rejects any summary number that no tool produced.

Table 2: Risk tiers for fixes (confirmed by the stakeholder, Table 1 item 2).

Fix	Risk	Handling
Trim spaces; merge labels that are identical after lowercasing and trimming	Low	Applied and logged
Parse dates in an unambiguous format; convert numbers stored as text (for example "\$1,200")	Low	Applied and logged
Remove exact duplicate rows	Medium	Ask the user
Exclude or flag rows (negative values, out of range, rule violations)	High	Ask the user
Fill missing values; change a number; merge labels by fuzzy match ("Elec." to "Electronics")	High	Ask the user

3.3 Tools and function calling

Every tool (Figure 1) has a JSON schema sent to the model and a Python function that does the work. Arguments are validated with pydantic first. A failed call returns the error to the model as text, for example "column 'Revnue' not found; columns are Order_ID, Date, ...", so it can correct itself instead of crashing the run, the same pattern we saw in the Codex source in Lab 5. Results stay short: counts and at most five example rows.

3.4 State, actions, observations and stopping

State is the profile, open issues, current version, change log, pending approvals, the request, and the step and cost budget used. **Actions** are tool calls, ask_user and finish. **Observations** are tool results, validator verdicts and user decisions. **Stopping:** the run ends when the planner calls finish with no failing checks, or when the budget (30 tool calls and a cost cap per task) runs out, in which case DataSage returns a partial report listing unresolved issues instead of guessing. No reward training is used; Section 6 scores only compare designs.

3.5 Memory and knowledge

Short-term memory is the session state, given to the model as a compact summary rebuilt each step instead of raw tool history. Knowledge comes from the check library (small documented functions the planner chooses from) and the optional rules file, read through `lookup_rule(column)`. Following the stakeholder's decision (Table 1, item 3), retrieval from trusted documentation is left as a later extension. As a Phase 4 goal, an approved cleaning plan can be saved as a "recipe" and replayed on next month's file with the same columns; this is our candidate for the reusable capability the course asks us to ship back.

3.6 One agent, not several

The task is sequential on one shared table, so a single planner agent avoids coordination cost without losing capability. The validator is plain code because row counts and totals must be checked exactly. If the planner misses problems, we will test an extra reviewer model call over the profile and change log.

3.7 Models, frameworks and error handling

We use Python 3.11+, pandas, openpyxl and pydantic with a function-calling model through an API; in Phase 2 we compare a small and a larger model and choose on accuracy first, then tool-call reliability and cost, since the stakeholder ranks correctness above cost and speed. We write the agent loop ourselves (a few hundred lines) rather than use a framework, to fully control state, the approval gate and

logging. The interface is a command line, plus a simple Streamlit page for the demo. **Error handling:** invalid arguments or tool exceptions go back to the model as text (at most two retries per action); failed validation rolls back and tells the model; sandbox code has a 30-second limit; API errors retry with backoff, then stop with a partial report. An action that fails twice is flagged and the planner moves on.

Section 4. Technical Challenges and Innovation

Much of DataSage is integration work: pandas, a model API and a command-line tool. Four parts go beyond that.

1. **Deciding between fix, flag and ask:** too many questions make the tool annoying; too few make it unsafe. Risk tiers are enforced in code, and we always report unsafe changes (target zero) together with questions per file, so one cannot be improved at the other's expense.
2. **Showing cleaning did not change meaning:** the validator compares each version cell by cell with the previous one, so any change no logged action explains is caught. This gives an exact count of unsupported changes, the failure mode named in the stakeholder's proposal.
3. **Keeping numbers honest:** language models can write numbers that sound right but are wrong. Every DataSage number comes from a tool, and the final check rejects a summary containing a number no tool produced.
4. **Tables larger than the context:** the model sees only a profile and a few sample rows, so choosing rows that reveal problems (such as rows that failed a check) is a real design problem. Ambiguous dates (03/04/2026), labels like "Elec." (so fuzzy merges are high risk) and negatives that may be refunds are further hard cases.

What is new. No single piece is new; the combination is. A model makes the judgment calls, while every change is versioned, checked, reversible and exported as plain pandas code, giving the user an answer plus an audit trail they can rerun without the agent.

Assumptions, risks and dependencies. Each file fits in memory with one header row (merged cells and multi-table sheets are out of scope). We depend on a model API, so every task has a cost and rate budget. Profiles and sample rows go to the API provider, so we use public or synthetic data unless the stakeholder agrees otherwise, with an option to hide sample values. The stakeholder agreed that public or synthetic messy datasets are fine for development and evaluation, so the project does not depend on receiving real files from him.

Section 5. Refined Baseline

B0: the stakeholder's baseline, kept unchanged. `run_baseline.py` loads a CSV or Excel file with pandas, runs five fixed checks (missing values, exact duplicate rows, negative numbers, invalid values in columns named "date" or "time", and labels that differ only by case), runs one fixed analysis (revenue by category) and writes a Markdown report. It never changes the data. We copied it unchanged into our repository (baseline/, commit 4b1df80) and ran it with Python 3.13 and pandas 3.0.5: it reproduced the proposal's numbers exactly and its smoke tests passed. It needs Python 3.11 or newer, because pandas 3 does.

Why we refine it. B0 does not clean, misses problems outside its rules (three of four in our probe file), and has one analysis, no approval and no validation. Comparing only against B0 would overstate our gain, since almost any cleaning beats it, so we add two simpler baselines that each test one idea (Table 3).

Table 3: Baselines.

Baseline	What it is	What the comparison shows
B0: stakeholder baseline (rule-based)	Fixed checks, no cleaning, one fixed analysis.	The gain over the stakeholder's current tool.

Baseline	What it is	What the comparison shows
B1: fixed cleaning script (rule-based, no model)	B0's checks plus fixes that are always applied: trim and lowercase labels, drop exact duplicates, set invalid dates to missing, drop rows with negative numbers. It receives each request already structured (group by, metric, aggregate, filter).	Whether per-file decisions beat a careful fixed script. B1 gets the structured request for free, so beating it is a fair test of the cleaning decisions.
B2: single model call	The same model as the agent, given the profile, sample rows and request in one prompt. It returns a cleaning plan and pandas code that we run once, with no checks, retries or approval.	Whether the loop, tools and validation add value over "just ask the model".

Section 6. Dataset and Evaluation Plan

Data:

1. **Planted-error benchmark (main test set):** clean public tables with clear reuse licenses (for example UCI or data.gov; each license recorded). A generator plants each in-scope error type at 1 to 5% of rows and records every planted error (row, column, type, original value). Each table gets two or three questions answered on the clean data. Target: about 40 tables and 120 tasks, split into a development set and a held-out test set.
2. **Real messy files:** five to ten files with unplanted problems, from public sources, plus any the stakeholder chooses to share, labeled separately by two team members who then settle disagreements.
3. **Fixtures:** the stakeholder's messy_sales files and our probe file, used in automated tests.

Privacy and licensing: we use only public or synthetic data and record each license. Any non-public stakeholder file stays out of the repository and is not sent to a model API without his permission.

Systems compared: B0, B1, B2 and DataSage, plus two ablations (no validator; every risky fix auto-approved) that show what each safety part adds.

Approvals during automated runs: an oracle built from the planted-error records approves a risky fix only if it repairs a planted error; a second run rejects every risky fix to check that DataSage stays correct when the user says no.

Table 4. Metrics and initial targets (agreed with the stakeholder as initial goals, Table 1 item 6).

Metric	How we measure it	Initial target
Final task performance		
Analysis accuracy	Answer matches the clean-data answer (each value within 0.5%)	≥ 90% of test tasks
Detection precision and recall	Planted problems found, matched by row, column and type	Both ≥ 0.85
Correction accuracy	Fixed cells equal the clean value	≥ 0.80 of planted errors to fix
Unsafe changes	Cells changed without a logged, approved reason, or valid cells damaged	0 unapproved destructive; < 1% of valid cells changed
Agent behavior		
Task completion	Answer and report produced within the budget	≥ 95%
Tool-call success	Valid tool calls / all tool calls (trace log)	≥ 95%
Recovery	Failures followed by a successful next step	≥ 80%
Questions per file	Approval prompts per task	Median ≤ 3
Latency	Wall-clock time per task	Median ≤ 60 s
Cost	Model API cost per task	Median ≤ \$0.10

What counts as meaningful improvement: on the held-out test set, DataSage should beat B1 and B2 on analysis accuracy by at least 15 percentage points, with zero unapproved destructive changes, while staying within the latency and cost targets. The stakeholder accepted these metrics and targets as initial goals, to be adjusted after baseline testing, and stressed that correctness matters more than latency and cost.

Human evaluation: after Phase 3 the stakeholder rates DataSage's reports on 10 tasks for correctness, clarity and trust (1 to 5 each); in Phase 4 he runs the acceptance test on files of his choice.

Plan for the rest of the semester:

- **Phase 2 (weeks 4 to 6):** profiler, check and fix tools, agent loop, error generator, first 40 tasks; run B0, B1 and B2.
- **Phase 3 (weeks 7 to 10):** approval gate, validator, sandbox, automated tests, evaluation harness, trace-log monitoring; second stakeholder meeting.
- **Phase 4 (weeks 11 to 13):** feedback on real files, the cleaning-recipe feature and the acceptance test; third stakeholder meeting.

Appendix A. Stakeholder Meeting Notes

Meeting: September 28, 2026, 4:00 to 5:30 PM, online over Zoom; Attendees: Vyshanth Buddani (stakeholder) and team members Shrey Bishnoi, Arsha Jindal, Ritik Agarwal, Vatsal Nirmal and Maulik Jadav.

Agenda and stakeholder answers:

- **Who would use DataSage, and on what kinds of files? Can you share two or three real messy files, or point us to public ones?**
Ans: DataSage is mainly intended for students, analysts, researchers, and small teams working with CSV/Excel files. Public or synthetic messy datasets can be used for development and evaluation.
- **Which problem types matter most to you, and which can wait?**
Ans: Focus first on missing values, duplicates, inconsistent labels, invalid dates/numbers, and other common tabular-data issues. More complex formats and ML can wait.
- **Which changes may DataSage make without asking, and which always need approval (Table 2)?**
Ans: Simple formatting fixes can be automatic and logged. Removing rows, changing/filling values, excluding suspicious data, or other ambiguous changes should require user approval.
- **What did you mean by retrieving external definitions? Would a rules file cover it (Table 1, item 3)?**
Ans: Start with a user-provided rules/data-definition file. Retrieval from trusted documentation can be an optional later extension.
- **Which analysis requests should it handle first?**
Ans: Prioritize filtering, grouping, aggregation, time summaries, and basic descriptive statistics.
- **Are about 60 seconds and \$0.10 per file acceptable?**
Ans: Around 60 seconds and \$0.10 per task are reasonable initial targets, but correctness is more important and these can be adjusted after testing.
- **Do you agree with the metrics and targets in Table 4? What would make you trust the report?**
Ans: The proposed metrics are suitable as initial targets. I would trust the system if its results were correct, the original data remains unchanged, every modification is logged, and uncertain changes require approval.

Action items: The team will keep the initial implementation focused on CSV/Excel, implement the approval and change-log workflow, evaluate against the baselines, and refine the numerical targets after testing. The stakeholder will review the working prototype and provide feedback.